Lancaster University Leipzig

# Inheritance vs Composition in OOP

LZMSCI521M | Week 1

Dr. Nono S. C. Merleau

# Why Object-Oriented Programming?

1. We model the world as objects: things with **state** and **behaviour**.
2. OOP helps us build bigger, safer programs by organising code.
3. You already use OOP daily: phone apps, games, web backends.

## Key Concept

1. Class = Mould, a **blueprint** for object, **attributes** + **methods**.
2. Object = Data (**state**) + Functions (**behavior**).
3. Why it matters: **encapsulation, reuse, polymorphism**.

## Today you will learn:

1. Objects vs Classes, state vs behaviours, and types of classes, examples in `Java`, `Python`, and `UML`.
2. Relationship between classes: Inheritance vs Composition

Lancaster University Leipzig

# 1.1

# OOP Concepts

Dr. Nono S. C. Merleau

University of Leipzig/ ScaDS.AI

# Notion Of Classes: Basic Syntax

Mould for building Objects:

1. **Attributes**: state representation, characteristics
2. **Methods**: behaviors, state controller.

```java
public class Rectangle{

  Pair<Integer, Integer> p1 = new Pair<Integer, Integer>(1, 2);
  Pair<Integer, Integer> p2 = new Pair<Integer, Integer>(3, 1);

  // Methods
  public int area() {
    int width = Math.abs(p1.getValue0() - p2.getValue0());
    int height = Math.abs(p1.getValue1() - p2.getValue1());
    return width * height;
  }
}
```

# Interfaces and Abstract Classes

- Interfaces are classes that contain methods without implementations
- Abstract classes are classes with at least one method without implementation

## Example

```java
public abstract class AbstractRectangle {
  Pair<Integer, Integer> p1 = new Pair<Integer, Integer>(1, 2);
  Pair<Integer, Integer> p2 = new Pair<Integer, Integer>(3, 1);

  //Abstract Methods
  public abstract int area();
}
```

# Implementating an Interface

```java
public interface IRectangle {

  public int area();
}
```

```java
public class RectangleImpl implements IRectangle {

  Pair<Integer, Integer> p1 = new Pair<Integer, Integer>(1, 2);
  Pair<Integer, Integer> p2 = new Pair<Integer, Integer>(3, 1);

  @Override
  public int area() {
    // TODO Auto-generated method stub
    return 0;
  }
}
```

# Objects

- Objects are instances of a class
- Objects can also represent different states of a class
- Objects are coherent entities that store data and the code (or instructions) working on that data.

## Example

```
Rectangle x = new Rectangle()
x.area()
```

```
Pair<Integer, Integer> p1 = Pair.with(5,2);
Pair<Integer, Integer> p2 = Pair.with(2, 3);
Rectangle y = new Rectangle(p1, p2);
```

# Instantiation (Constructors)

```java
public class Rectangle{
  Pair<Integer, Integer> p1 = Pair.with(1, 2);
  Pair<Integer, Integer> p2 = Pair.with(3, 1);

  //Constructor
  public Rectangle(Pair<Integer, Integer>p1, Pair<Integer,
   Integer> p2) {
    this.p1= p1;
    this.p2 = p2;
  }
  // Methods
  public int area() {
    int width = Math.abs(p1.getValue0() - p2.getValue0());
    int height = Math.abs(p1.getValue1() - p2.getValue1());
    return width * height;
  }}
```

1.2

# Inheritance vs Composition in OOP

Dr. Nono S. C. Merleau

University of Leipzig/ ScaDS.AI

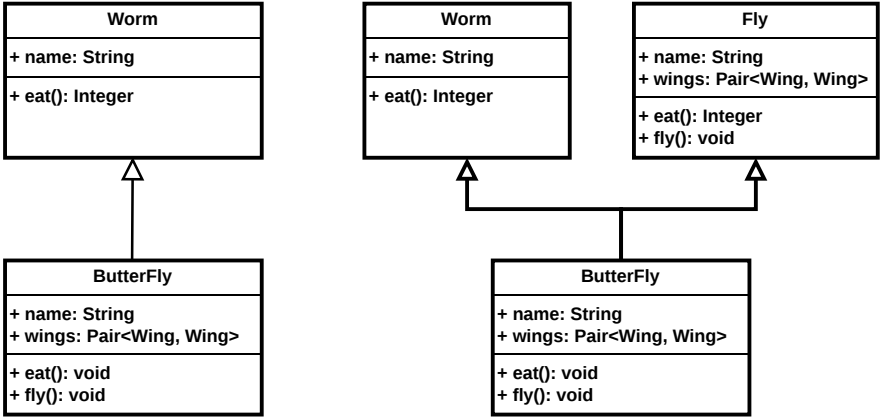
# OOP: Inheritance (is-a)

- A subclass inherit from a base class to **reuse** and **specialise** its parents' attributes and methods.
- Classes inherited from another are derived classes, **subclasses**, or **subtypes**.
- Classes from which other classes are derived are called base classes or **superclasses**.
- Use when the subclass truly **is-a** base class and respects the **LSP** (Liskov Subtitution Principle).

```java
public class Square extends Rectangle {}
```

```java
Square square = new Square();
System.out.println(square); //Rectangle([1, 2], [3, 1])
```

# OOP: Multiple inheritance




- **Conflict of method names**
- **Repetition or duplication of information**

# OOP: Multiple inheritance and Java

A class can be derived from more than one superclass in Python.

```
class Worm:
  def __init__ (self, name):
    self.name = name
  def eat(self):
    print(self.name +" swallows")
class Fly:
  def __init__ (self, name):
    self.name = name
  def eat(self):
    print(self.name +" is nibbling..")
class ButterFly(Worm, Fly):
  pass
```

**In Java**: How can we implement it? (Home work (1))

# OOP: Composition (has-a, delegation)

Composition is an OOP concept that models the relationship **has-a**.

- **Composite class**: **has a** components and **delegates** work to them.
- **Component class**: part of the composite class

**Example**: We already used it in the previous Example, but how and where? (Home work (2))

## Remarks

- More **flexible** compared to inheritance
- Changes to the component class have minimal or no effects on the composite class
- Better **encapsulation**, but a bit more wiring.

# Example of Composition in Java

```java
public class Fly {
  public String name;
  public Pair<Wing, Wing> wings;

  public Fly(String name, Wing left, Wing right) {
    this.name = name;
    this.wings = Pair.with(left, right);
  }

  public void eat() {
    System.out.println(this.name +" is nibbling...");
  }
  public void fly() {
    System.out.println("Hi! I believe I can fly. ");
  }}
```

# Example of Composition in Java

```java
public class Wing {
  public String name = null;
  public Integer length = 4;
  public String wingColor = "white";

  public Wing() {}

  public Wing(String name, Integer length, String color) {
    this.name = name ;
    this.length = length;
    this.wingColor = color;
  }
}
```

# 1.3

# Takeaway

Dr. Nono S. C. Merleau

University of Leipzig/ ScaDS.AI

# Composition vs Inheritance (when to choose which)

**Inheritance (is-a)**

- *Pros*: Simple reuse, polymorphism via base type.
- *Cons*: Tight coupling to base; fragile base class; LSP risks.
- *Use when*: True subtype, stable base API, shared behaviour.

**Composition (has-a)**

- *Pros*: Replaceable parts, multiple change axes, better encapsulation.
- *Cons*: More wiring; no automatic access to base internals.
- *Use when*: Need variability, combine features, avoid deep trees.

**Remarks:** Prefer composition for flexibility; use inheritance for true subtyping with stable behaviour.

1.4

# Home Work 1.0

Dr. Nono S. C. Merleau

University of Leipzig/ ScaDS.AI

# Questions

1. Consider the Multiple inheritance UML class diagram on **Slide 18**:
   a. Use the nation of Interface to implement that diagram
   b. How can we solve the conflict of method names in `Python`?
2. Given the `Java` Class Rectangle in **Slide 11**:
   a. Find where the composition is being used
   b. Can we replace that by an Inheritance relationship?
   c. How can this example illustrate the polymorphism?
3. When would **inheritence** still be the cleaner choice?

# 1.5

# What Next?

Dr. Nono S. C. Merleau

University of Leipzig/ ScaDS.AI

# What next?

Encapsulation, Polymorphism, and other Class relationships, e.g., association, aggregation, etc...

# Beyond OOP

1. Domain Driven Design (DDD)
2. Agent Oriented Programming (AOP)
3. Design Paterns

## Some Important Materials

- Bloch, Joshua. Effective Java. 3rd ed. Addison-Wesley, 2018.
- Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- Code and Course Material